6

Getting Your Hands Dirty with Unity

Creating a game involves much more than just simulating actions in code. Design, story, environment, lighting, and animation all play an important part in setting the stage for your players. A game is, first and foremost, an experience—something that code alone can't deliver.

Unity has placed itself at the forefront of game development over the past decade by bringing advanced tools to programmers and non-programmers alike. Animation and effects, audio, environment design, and much more are all available directly from the Unity Editor without a single line of code. We'll discuss these topics as we define the requirements, environment, and game mechanics of our game. However, first, we need a topical introduction to game design.

Game design theory is a large area of study and learning all its secrets can consume an entire career. However, we'll only be getting hands-on with the basics; everything else is up to you to explore! This chapter will set us up for the rest of the book and will cover the following topics:

- A game design primer
- Building a level
- Lighting basics
- Animating in Unity

A game design primer

Before jumping into any game project, it's important to have a blueprint of what you want to build. Sometimes, ideas will start crystal clear in your mind, but the minute you start creating character classes or environments, things seem to drift away from your original intention. This is where the game's design allows you to plan out the following touchpoints:

- **Concept**: The big-picture idea and design of a game, including its genre and play style.
- **Core mechanics**: The playable features or interactions that a character can take in-game. Common gameplay mechanics include jumping, shooting, puzzle-solving, or driving.
- **Control schemes**: A map of the buttons and/or keys that give players control over their character, environment interactions, and other executable actions.
- **Story**: The underlying narrative that fuels a game, creating empathy and a connection between players and the game world they play in.
- **Art style**: The game's overarching look and feel, consistent across everything from characters and menu art to the levels and environments.
- **Win and lose conditions**: The rules that govern how the game is won or lost, usually consisting of objectives or goals that carry the weight of potential failure.

These topics are by no means an exhaustive list of what goes into designing a game. However, they're a good place to start fleshing out something called a **game design document**, which is your next task!

Game design documents

Googling `game design documents` will result in a flood of templates, formatting rules, and content guidelines that can leave a new programmer ready to give it all up. The truth is, design documents are tailored to the team or company that creates them, making them much easier to draft than the internet would have you think.

In general, there are three types of design documentation, as follows:

- **Game Design Document (GDD)**: The GDD houses everything from how the game is played to its atmosphere, story, and the experience it's trying to create. Depending on the game, this document can be a few pages long or several hundred.
- **Technical Design Document (TDD)**: This document focuses on all the technical aspects of the game, from the hardware it will run on to how the classes and program architecture need to be built out. Like a GDD, the length will vary based on the project.

- **One-Page**: Usually used for marketing or promotional situations, a one-page document is essentially a snapshot of your game. As the name suggests, it should only take up a single page.

There's no right or wrong way to format a GDD, so it's a good place to let your brand of creativity thrive. Throw in pictures of reference material that inspires you; get creative with the layout—this is your place to define your vision.

The game we'll be working on throughout the rest of this book is fairly simple and won't require anything as detailed as a GDD or TDD. Instead, we'll create a one-pager to keep track of our project objectives and some background information.

The Hero Born one-pager

To keep us on track going forward, I've put together a simple document that lays out the basics of the game prototype. Read through it before moving on, and try to start imagining some of the programming concepts that we've learned so far being put into practice:

Concept
Game prototype focused on stealthily avoiding enemies and collecting health items - with a little FPS on the side.

Gameplay
Main mechanic centers around using line-of-sight to stay one step ahead of patrolling enemies and collecting required items.

Combat will consist of shooting projectiles at enemies, which will automatically trigger an attack response.

Interface
Control scheme for movement will be the WASD or arrow keys using the mouse for camera control. Shooting mechanic will use the Space bar, and item collection will work off of object collisions.

Simple HUD will show items collected and remaining ammo, as well as a standard health bar.

Art Style
Level and character art style will be all primitive GameObjects for fast and efficient, no-frills development. These can be swapped out at a later date with 3D models or terrain environments if needed.

Figure 6.1: Hero Born one-page document

To provide a complete view of the Unity editor, all our screenshots are taken in full-screen mode. For color versions of all book images, use the link below: `https://packt.link/7yy5V`.

Now that you have a high-level view of the pillars of our game, you're ready to start building a prototype level to house the game experience.

Building a level

When building your game levels, it's always a good idea to try to see things from the perspective of your players. How do you want them to see the environment, interact with it, and feel while walking around in it? You're literally building the world your game exists in, so be consistent.

With Unity, you can use basic 3D shapes to block out simple environments, the more advanced **ProBuilder tool**, or a mixture of the two. You can even import 3D models from other programs, such as Blender, to use as objects in your scenes.

Unity has a great introduction to the ProBuilder tool at: `https://unity.com/features/probuilder`.

You can also use tools like Blender to create your game assets, which you can find at: `https://www.blender.org/features/modeling/`.

For *Hero Born*, we'll stick with a simple indoor arena-like setting that's easy to get around, but with a few corners to hide in. You'll cobble all this together using **primitives**—base object shapes provided in Unity—because of how easy they are to create, scale, and position in a scene.

Creating primitives

Looking at games you might play regularly, you're probably wondering how you'll ever create models and objects that look so realistic that it seems you could reach through the screen and grab them. Fortunately, Unity has a set of primitive GameObjects that you can select from to prototype faster. These won't be super fancy or high-definition, but they are a lifesaver when you're learning the ropes or don't have a 3D artist on your development team.

If you open up Unity, you can go into the **Hierarchy** panel and click on **+ > 3D Object**, and you'll see all the available options, but only about half of these are primitives or common shapes, indicated in the following screenshot:

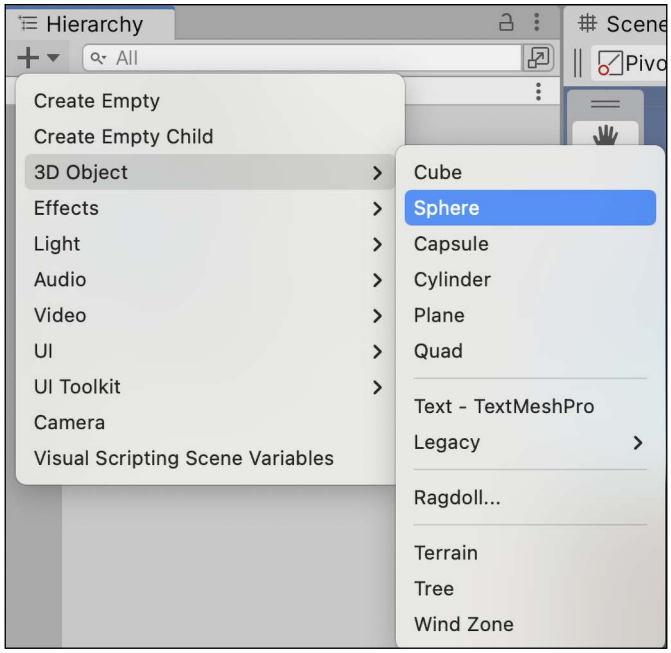

Figure 6.2: Unity Hierarchy window with the 3D Object option selected

Other **3D Object** options, such as **Terrain**, **Wind Zone**, and **Tree**, are a bit too advanced for what we need, but feel free to experiment with them if you're interested.

You can find out more about building Unity environments at: `https://docs.unity3d.com/Manual/CreatingEnvironments.html`.

Before we jump too far ahead, it's usually easier to walk around when you've got a floor underneath you, so let's start by creating a ground plane for our arena using the following steps:

1. In the **Hierarchy** panel, click on **+** > **3D Object** > **Plane**.
2. Select the new object in the **Hierarchy** tab and rename the GameObject to `Ground` in the **Inspector** tab or by pressing *Enter*.

3. In the **Transform** dropdown, change **Scale** to 3 in the **X**, **Y**, and **Z** axes:

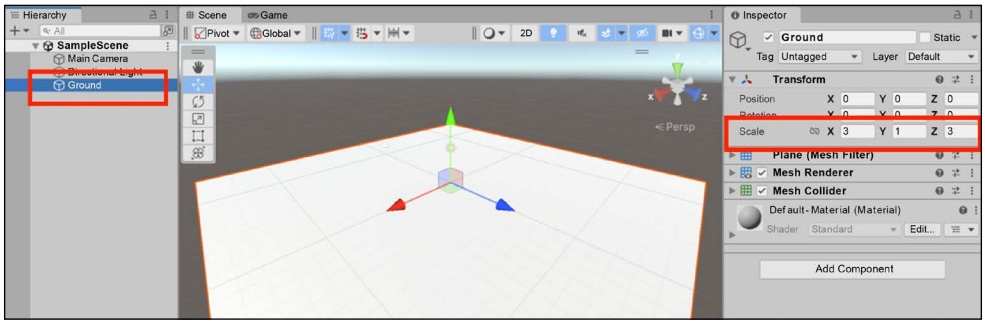

Figure 6.3: Unity Editor with a Ground plane

🔍 **Quick tip**: Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

🔓 **The next-gen Packt Reader** and a **free PDF/ePub copy** of this book are included with your purchase. Unlock them by scanning the QR code below or visiting `https://www.packtpub.com/unlock/9781837636877`.

4. If the lighting in your scene looks dimmer or different from the preceding image, select **Directional Light** in the **Hierarchy** panel, and set the **Intensity** value of the **Directional Light** component to 1:

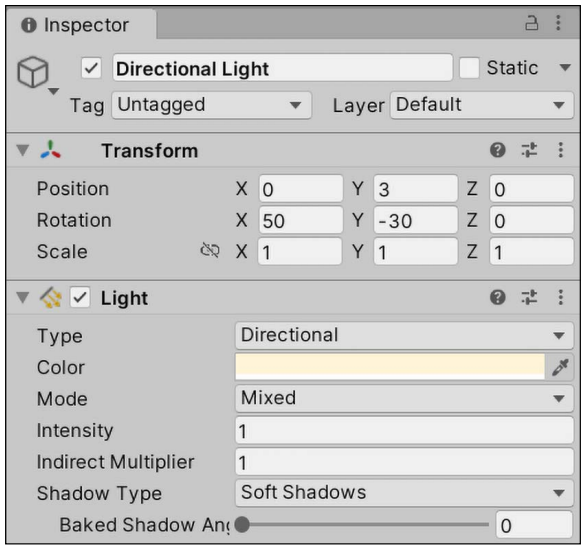

Figure 6.4: Directional Light object selected in the Inspector pane

Here, we created a plane GameObject, and increased its size to make more room for our future characters to walk around. This plane will act like a 3D object bound by real-life physics, meaning other objects will know it's there and won't just fall through the floor into oblivion. We'll talk more about the Unity physics system and how it works in *Chapter 7*, *Movement, Camera Controls, and Collisions*. Right now, we need to start thinking in 3D.

Thinking in 3D

Now that we have our first object in the scene, we can talk about 3D space—specifically, how an object's position, rotation, and scale behave in three dimensions. If you think back to high school geometry, a graph with an *x* and *y* coordinate system should be familiar. To put a point on the graph, you need an *x* value and a *y* value.

Unity supports both 2D and 3D game development, and if we were making a 2D game, we could leave our explanation there. However, when dealing with 3D space in the Unity Editor, we have an extra axis, called the *z* axis. The *z* axis maps depth, or perspective, giving our space and the objects in it their 3D quality.

This might be confusing at first, but Unity has some nice visual aids to help you get your head on straight. In the top-right corner of the **Scene** panel, you'll see a geometric-looking icon with the *x*, *y*, and *z* axes marked in red, green, and blue, respectively. All GameObjects in the scene will show their axis arrows when they're selected in the **Hierarchy** window:

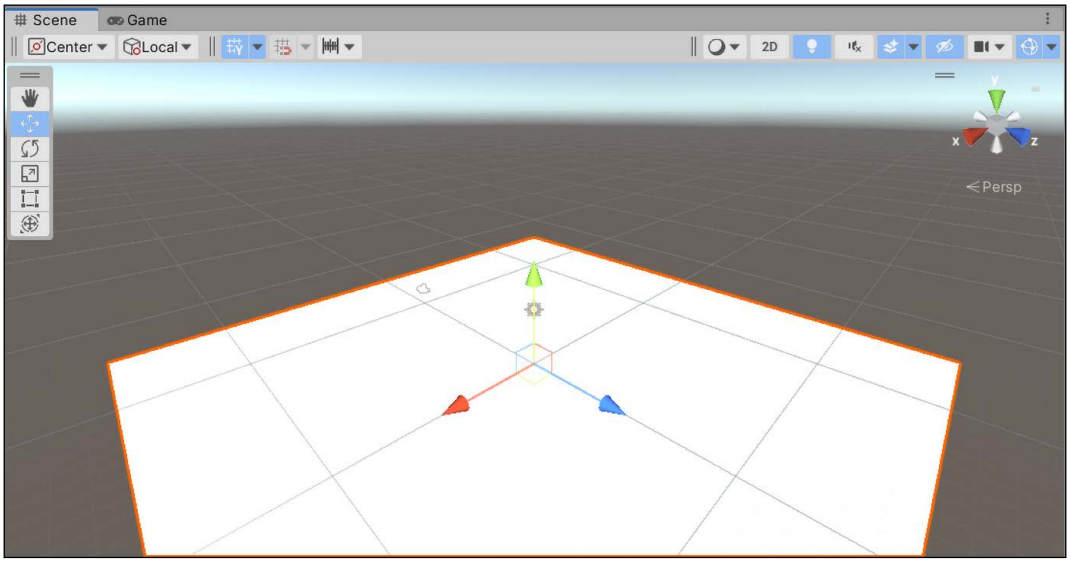

Figure 6.5: Scene view with the orientation gizmo highlighted

This will always show the current orientation of the scene and the objects placed inside it. Clicking on any of the colored axes will switch the scene orientation to the selected axis. Give this a go yourself to get comfortable with switching perspectives.

If you take a look at the **Ground** object's **Transform** component in the **Inspector** window, you'll see that the position, rotation, and scale are all determined by these three axes.

The position determines where the object is placed in the scene, its rotation governs how it's angled, and its scale takes care of its size. These values can be changed at any time in the **Inspector** pane or in a C# script:

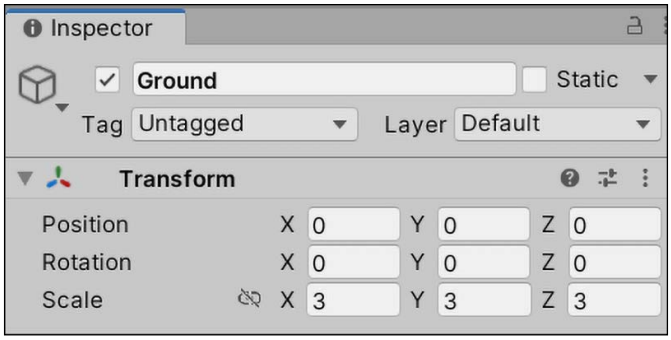

Figure 6.6: Ground object selected in the Hierarchy

Right now, the ground is looking a little boring. Let's change that with a material.

Materials

Our ground plane isn't very interesting right now, but we can use **Materials** to breathe a little life into the level. Materials are in charge of setting GameObject properties like color and texture; the material is passed to the shader, which uses the shader to render the material properties onscreen. Think of **Shaders** as being responsible for combining lighting and texture data into a representation of how the material looks.

Each GameObject starts with a default **Material** and **Shader** (pictured here from the **Inspector** pane), setting its color to a standard white:

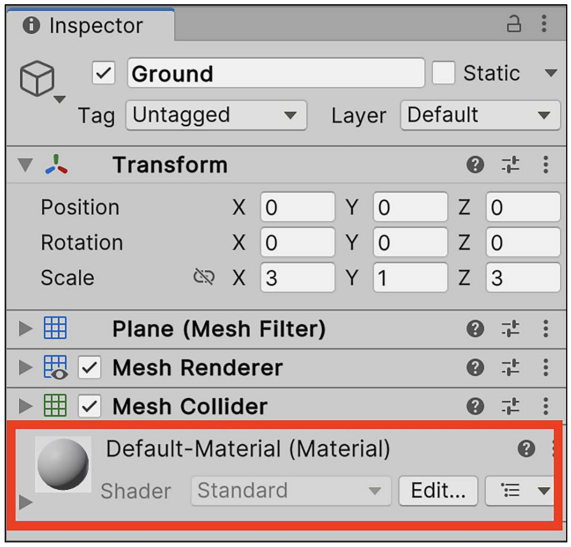

Figure 6.7: Default material on an object

To change an object's color, we need to create a material and drag it to the object that we want to modify. Remember, everything is an object in Unity—materials are no different. Materials can be reused on as many GameObjects as needed, but any change to a material will also carry through to any objects the material is attached to. If we had several enemy objects in the scene with a material that set them all to red, and we changed that base material color to blue, all our enemies would then be blue.

Blue is eye-catching; let's change the color of the ground plane to match, and create a new material to turn the ground plane from a dull white to a dark and vibrant blue:

1. Create a new folder in the **Project** panel by *right-clicking* > **Create** > **Folder**, and name it `Materials`.
2. Inside the **Materials** folder, *right-click* > **Create** > **Material**, and name it `Ground_Mat`.
3. Select the new material in the **Project** panel and look at its properties in the **Inspector**. Click on the color box next to the **Albedo** property, select your color from the color picker window that pops up (the big square in the middle sets the actual color), and then close it:

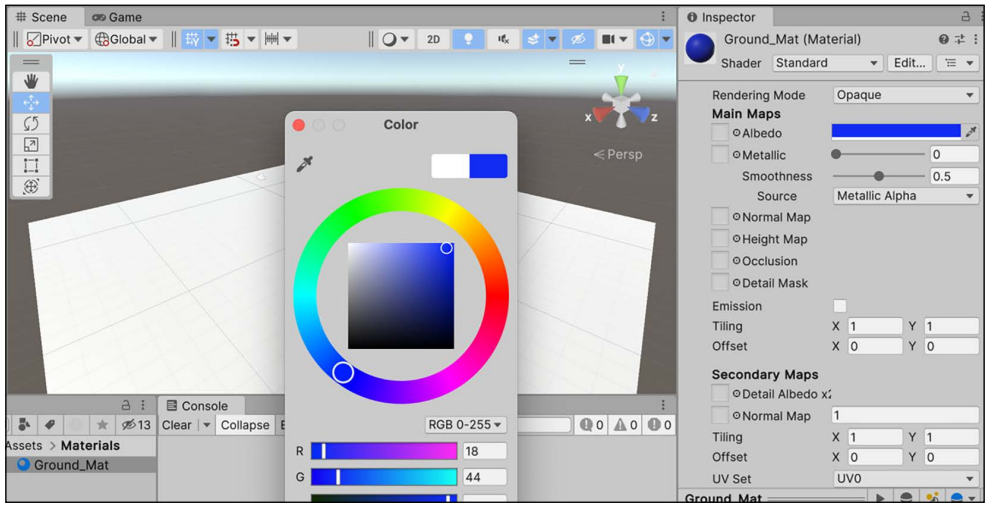

Figure 6.8: Material color picker

4. Drag the `Ground_Mat` object from the **Project** panel, and drop it onto the `Ground` GameObject in the **Hierarchy** panel.

The new material you created is now a project asset. Dragging and dropping `Ground_Mat` from the **Project** panel onto the `Ground` GameObject in the **Hierarchy** changed the color of the plane, which means any changes to `Ground_Mat` will be reflected in the `Ground` GameObject.

If you select `Ground` in the **Hierarchy**, you'll also see the **Ground_Mat (Material)** component at the very bottom is now being used by the **Standard Shader** to render the plane's color property:

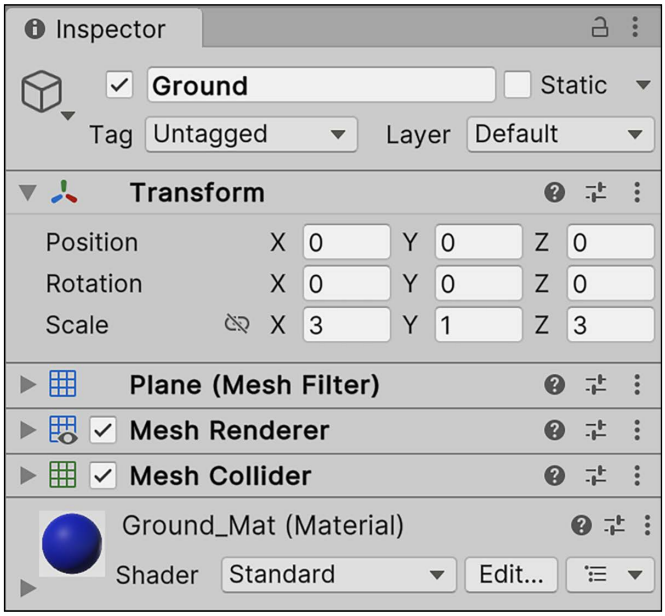

Figure 6.9: Ground plane with the updated color material

The ground is our canvas; however, in 3D space, it can support other 3D objects on its surface. It'll be up to you to populate it with fun and interesting obstacles for your future players, which we'll do in the next section when we learn a little about white-boxing!

White-boxing

White-boxing is a design term for laying out ideas using placeholders, usually with the intent of replacing them with finished assets at a later date. In level design, the practice of white-boxing is to block out an environment with primitive GameObjects to get a sense of how you want it to look. This is a great way to start things off, especially during the prototyping stages of your game.

Before diving into Unity, I'd like to start with a simple sketch of the basic layout and position of my level. This gives us a bit of direction and will help to get our environment laid out quicker.

In the following drawing, you'll be able to see the arena I have in mind, with a raised platform in the middle that is accessible by ramps, complete with small turrets in each corner:

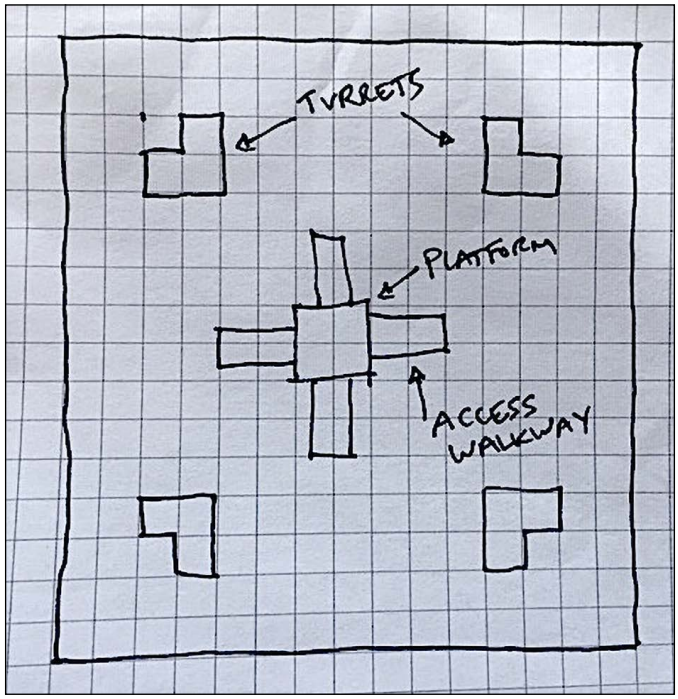

Figure 6.10: Sketch of the Hero Born level arena

Don't worry if you're not an artist—neither am I. The important thing is to get your ideas down on paper to solidify them in your mind and work out any kinks before getting busy in Unity.

Before you go full steam ahead and put this sketch into production, you'll need to familiarize yourself with a few Unity Editor shortcuts to make white-boxing easier.

Editor tools

When we discussed the Unity interface in *Chapter 1*, *Getting to Know Your Environment*, we skimmed over some of the Toolbar functionality, which we need to revisit so that we know how to efficiently manipulate GameObjects. You can find these in the upper-left corner of the Unity Editor:

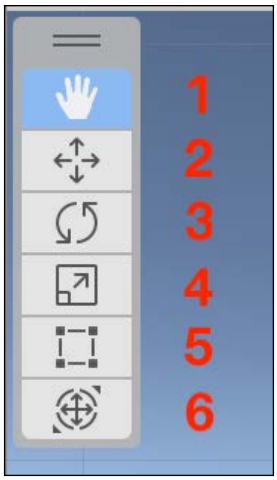

Figure 6.11: Unity Editor Toolbar

Let's break down the different tools that are available to us from the Toolbar in the preceding screenshot:

1. **View**: This allows you to pan and change your position in the scene by clicking and dragging your mouse.
2. **Move**: This lets you move objects along the *x*, *y*, and *z* axes by dragging their respective arrows.
3. **Rotate**: This lets you adjust an object's rotation by turning or dragging its respective markers.
4. **Scale**: This lets you modify an object's scale by dragging it to specific axes.
5. **Rect Transform**: This combines the **Move**, **Rotate**, and **Scale** tool functionality into one package.
6. **Transform**: This gives you access to the position, rotation, and scale of an object all at once.

You can find more information about navigating and positioning GameObjects in the **Scene** panel at: `https://docs.unity3d.com/Manual/PositioningGameObjects.html`. It's also worth noting that you can move, position, and scale objects using the **Transform** component, as we discussed earlier in the *Thinking in 3D* section.

Panning and navigating the scene can be done with similar tools, although not from the Unity Editor itself:

- To look around, hold down the right mouse button and drag it to pan the camera around.
- To move around while using the camera, continue to hold the right mouse button and use the *W*, *A*, *S*, and *D* keys to move forward, back, left, and right, respectively. If you're on a Mac and using your trackpad instead of a mouse, click-and-hold two fingers while pressing the *W*, *A*, *S*, and *D* keys.
- Hit the *F* key to zoom in and focus on a GameObject that has been selected in the **Hierarchy** panel.

This kind of scene navigation is more commonly known as fly-through mode, so when I ask you to focus on or navigate to a particular object or viewpoint, use a combination of these features.

Getting around the **Scene** view can be a task all on its own, but it all comes down to repeated practice. For a more detailed list of scene navigation features, visit: `https://docs.unity3d.com/Manual/SceneViewNavigation.html`.

Even though the ground plane won't allow our character to fall through it, we could still walk off the edge at this point. Now, your job is to wall in the arena so that the player has a confined locomotion area.

Hero's trial—putting up drywall

Using primitive cubes and the toolbar, position four walls around the level using the **Move**, **Rotate**, and **Scale** tools to section off the main arena:

1. In the **Hierarchy** panel, select **+ > 3D Object > Cube** to create the first wall and name it `Wall`.
2. Set its scale value to `30` for the *x* axis, `1.5` for the *y* axis, and `0.2` for the *z* axis.

Note that planes operate on a scale 10 times larger than objects—so our plane with a length of 3 is the same length as any other object of length 30.

3. With the `Wall` object selected in the **Hierarchy** panel, switch to the **Position** tool in the upper-left corner and use the red, green, and blue arrows to position the wall at the edge of the ground plane.
4. Repeat *steps 1-3* until you have four walls surrounding your area:

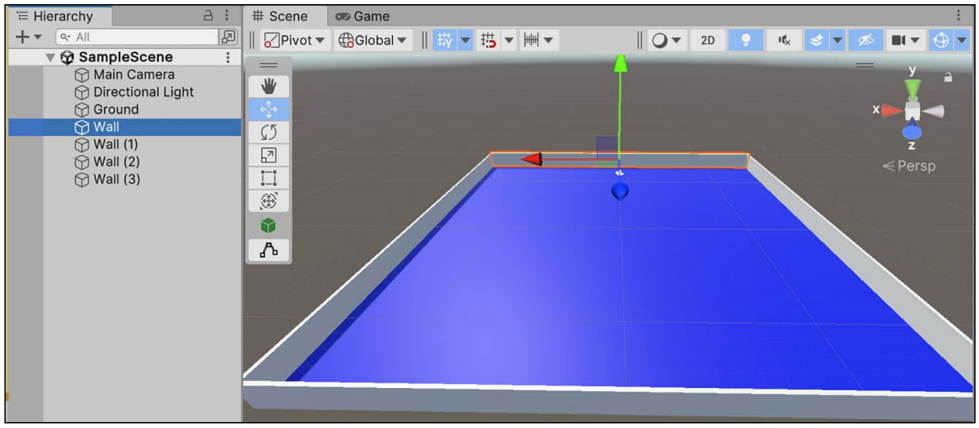

Figure 6.12: Level arena with four walls and a ground plane

From this chapter onward, I'll be giving some basic values for wall position, rotation, and scale, but feel free to be adventurous and use your own creativity. I want you to experiment with the Unity Editor tools so you get comfortable faster.

That was a bit of construction, but the arena is starting to take shape! Before we move on to adding obstacles and platforms, you'll want to get into the habit of cleaning up your object hierarchy. We'll talk about how that works in the following section.

Keeping the hierarchy clean

Normally, I would put this sort of advice in a blurb at the end of a section, but making sure your project hierarchy is as organized as possible is so important that it needs its own subsection. Ideally, you'll want all related GameObjects to be under a single **parent object**. Right now, it's not a risk because we only have a few objects in the scene; however, when that gets into the hundreds on a big project, you'll be struggling.

The easiest way to keep your hierarchy clean is to store related objects in a parent object, just as you would with files inside a folder on your desktop. Our level has a few objects that could use some organization, and Unity makes this easy by letting us create empty GameObjects. An empty object is a perfect container (or folder) for holding related groups of objects because it doesn't come with any components attached—it's a shell.

Let's take our ground plane and four walls and group them all under a common empty GameObject:

1. Select **+** > **Create Empty** in the **Hierarchy** panel and name the new object `Environment`.
2. Select the **Environment** empty object and check that its **X**, **Y**, and **Z** positions are all set to 0.
3. Drag and drop the ground plane and the four walls into **Environment**, making them child objects:

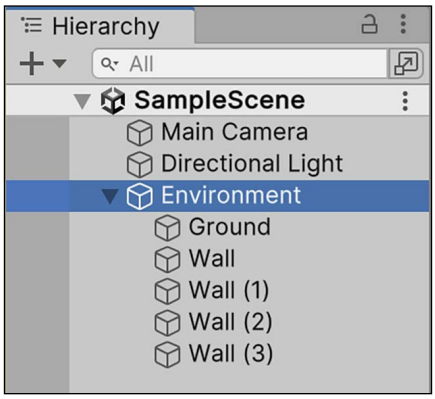

Figure 6.13: Hierarchy panel showing the empty GameObject parent

The environment exists in the **Hierarchy** tab as a parent object, with the arena objects as its children. Now we're able to expand or close the **Environment** object drop-down list with the arrow icon, making the **Hierarchy** panel less cluttered.

It's important to set the **Environment** object's **X**, **Y**, and **Z** positions to `0` because the child object positions are now relative to the parent position. This leads to an interesting question: what are the origin points of these positions, rotations, and scales that we're setting? The answer is that they depend on what relative space we're using, which, in Unity, is either **World** or **Local**:

- **World space** uses a set origin point in the scene as a constant reference for all GameObjects. In Unity, this origin point is (`0`, `0`, `0`), or `0` on the x, y, and z axes.
- **Local space** uses the object's parent `Transform` component as its origin, essentially changing the perspective of the scene. Unity also sets this local origin to (`0`, `0`, `0`). Think of this as the parent `Transform` being the center of the universe, with everything else orbiting in relation to it.

Both of these orientations are useful in different situations, but right now, resetting it at this point starts everyone on an even playing field.

Working with Prefabs

Prefabs are one of the most powerful components you'll come across in Unity. They come in handy not only in level building but in scripting as well. Think of Prefabs as GameObjects that can be saved and reused with every child object, component, C# script, and property setting intact. Once created, a Prefab is like a class template; each copy used in a scene is a separate instance of that Prefab. Consequently, any change to the base Prefab will also change all the unaltered active instances in the scene.

The arena looks a little too simple and completely wide open, making it a perfect place to test out creating and editing Prefabs. Since we want four identical turrets in each corner of the arena, they're a perfect case for a Prefab.

Again, I haven't included any precise barrier position, rotation, or scale values because I want you to get up close and personal with the Unity Editor tools.

Going forward, when you see a task ahead of you that doesn't include specific position, rotation, or scale values, I'm expecting you to learn by doing.

Let's create the turrets with the following steps:

1. Select the **Environment** parent object, **right-click > Create Empty**, and name it `Barrier`.
2. Select **Barrier** and **right-click > 3D Object > Cube** twice, then position and scale them as a v-shaped base.
    a. Change the **X Scale** of the first **Cube** to 3 and the second cube to 4 if you're not sure how big they should be.

3. Create two more cube primitives (no need to rescale them) and place them on top of the ends of the turret base:

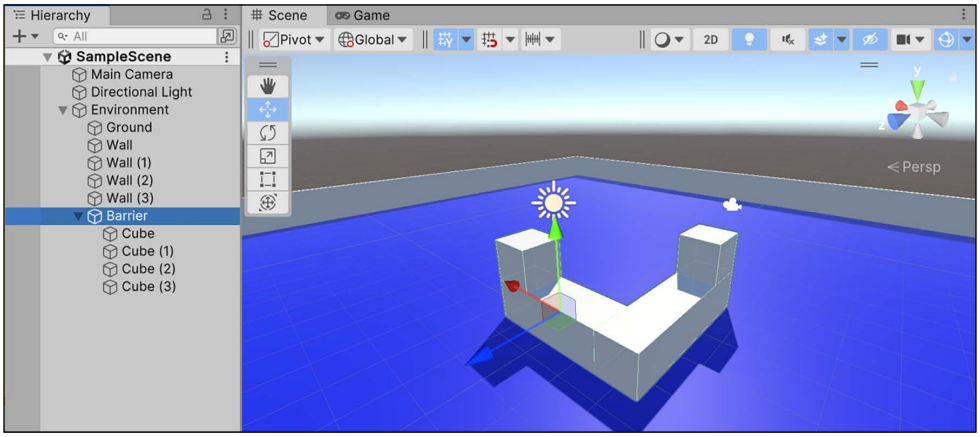

Figure 6.14: Screenshot of the turret composed of cubes

4. In the main **Assets** folder, create a new folder named `Prefabs` and drag the **Barrier** GameObject from the **Hierarchy** panel to the **Prefabs** folder in the **Project** view:

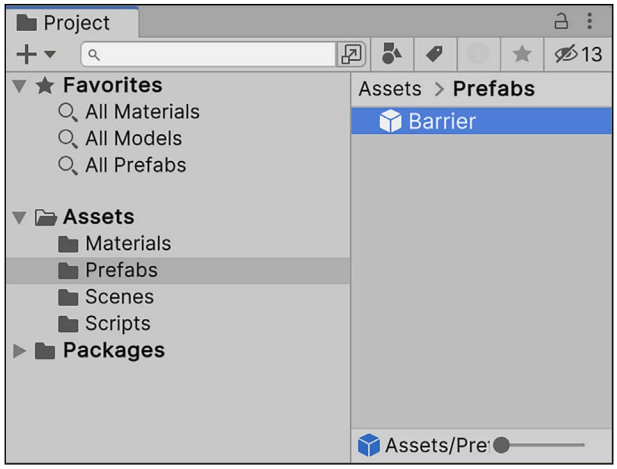

Figure 6.15: Barrier Prefab in the Prefabs folder

Barrier, and all its child objects, are now Prefabs, meaning that we can reuse it by dragging copies from the `Prefabs` folder or duplicating the one in the scene. **Barrier** turned blue in the **Hierarchy** tab to signify its status change, and also added a row of **Prefab** function buttons in the **Inspector** tab underneath its name:

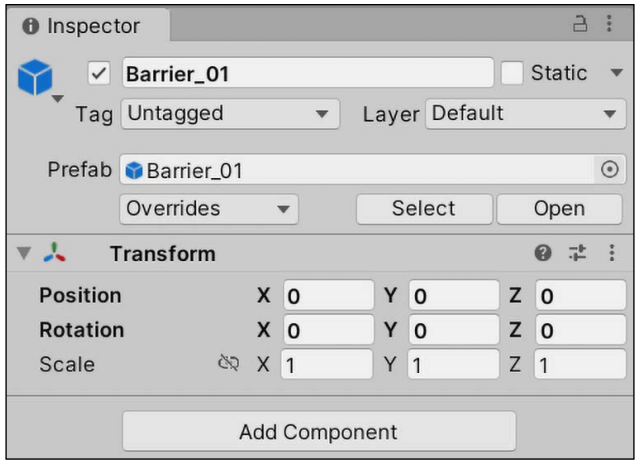

Figure 6.16: Barrier_01 Prefab highlighted in the Inspector pane

Any edits to the original Prefab object, **Barrier**, will now affect any copies in the scene. Since we need a fifth cube to complete the barrier, let's update and save the Prefab to see this in action.

Now our turret has a huge gap in the middle, which isn't ideal for covering our character, so let's update the **Barrier** Prefab by adding another cube and applying the change:

1. Create a **Cube** primitive and place it at the intersection of the turret base.
    a. The new **Cube** primitive will be marked as gray with a little + icon next to its name in the **Hierarchy** tab. This means it's not officially part of the Prefab yet:

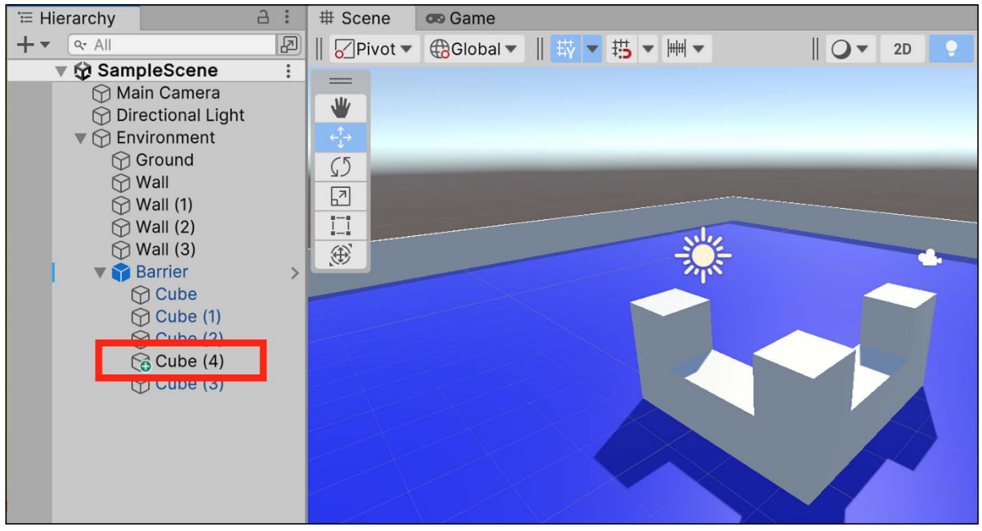

Figure 6.17: New Prefab update marked in the Hierarchy window

2. Right-click on the new **Cube** primitive in the **Hierarchy** panel and select **Added GameObject > Apply to Prefab 'Barrier'**:

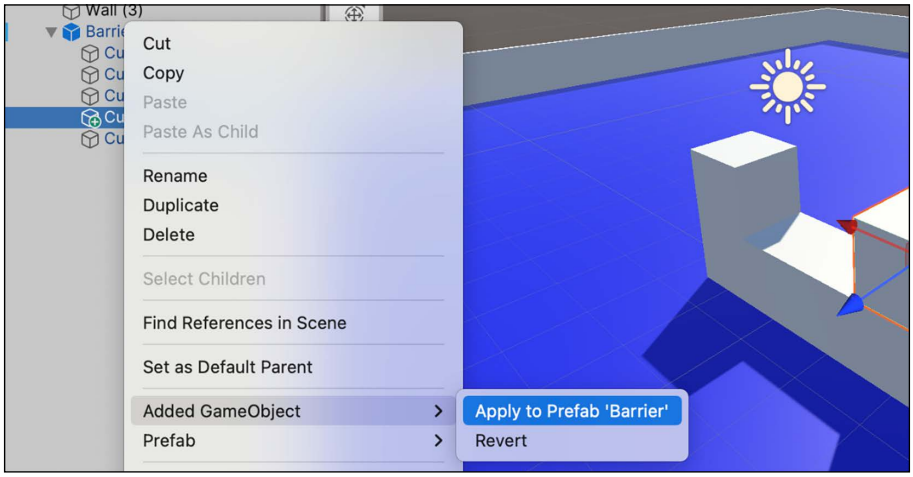

Figure 6.18: Option to apply Prefab changes to the base Prefab

The **Barrier** Prefab is now updated to include the new cube, and the entire Prefab hierarchy should be blue again. You now have a turret Prefab that looks like the preceding screenshot or, if you're feeling adventurous, something more creative. However, we want these to be in every corner of the arena.

Duplicate the **Barrier** Prefab three times and place each one in a different corner of the arena. You can do this by dragging multiple **Barrier** objects from the **Prefabs** folder into the scene, or right-clicking on **Barrier** in the **Hierarchy** and selecting **Duplicate**.

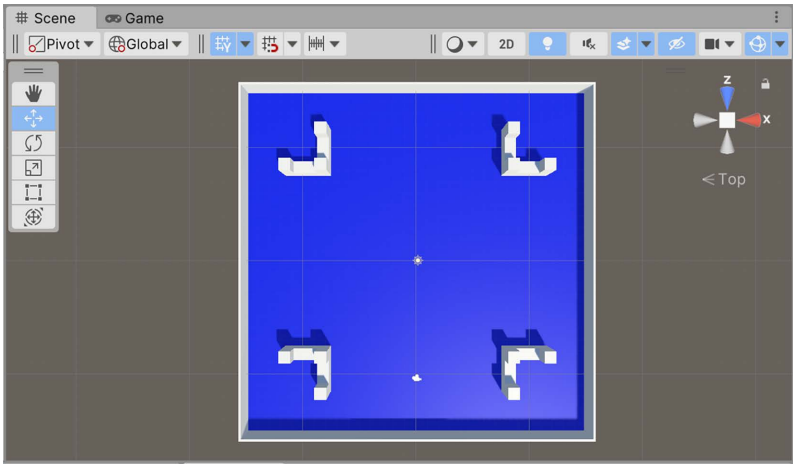

Figure 6.19: Barriers placed in the corners of the arena

Now that we've got a reusable barrier Prefab, let's build out the rest of the level to match the rough sketch that we had at the beginning of the section:

1. Create a new empty GameObject inside the **Environment** parent object, name it `Platform`, and set its position in the *x*, *y*, and *z* axes to `0`.
2. Create a **Cube** as a child object of `Platform` and scale it (*x*: 5, *y*: 2, *z*: 5) to form a platform as shown in *Figure 6.20* below.
3. Create a **Plane** and scale it into a ramp (*x*: 1, *y*: 1, *z*: 0.5):
    - Hint: Rotate the plane around the *z* axis to create an angled plane
    - Then, position it so that it connects the platform to the ground with enough room to walk onto it
4. Duplicate the ramp object by using *Cmd* + *D* on a Mac, or *Ctrl* + *D* on Windows. Then, repeat the rotation and positioning steps.
5. Repeat the previous step twice more, until you have four ramps in total leading to the platform:

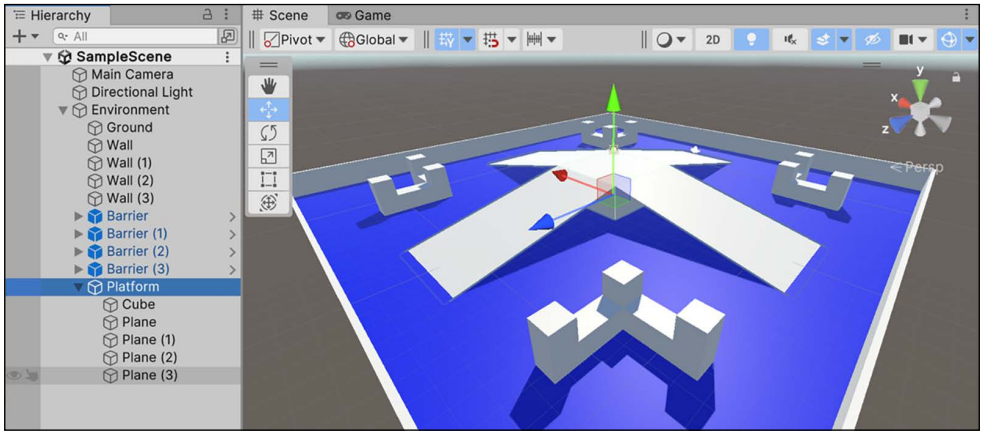

Figure 6.20: Platform parent GameObject

You've now successfully white-boxed your first game level! Don't get too caught up in it yet, though—we're just getting started. All good games have items that players can pick up or interact with. In the following challenge, it's your job to create a health item and make it a Prefab.

Hero's trial—creating a health pickup

Putting together everything we've learned so far in this chapter might take you a few minutes, but it's well worth the time. Create the pickup item as follows:

1. Create a **Capsule** GameObject by selecting **+** > **3D Object** > **Capsule** and name it `Health_Pickup`.
2. Set the scale to `0.3` for the *x*, *y*, and *z* axes, and then switch to the **Move** tool and position it near one of your barriers.
3. Create and attach a new yellow-colored **Material** to the **Health_Pickup** object.
4. Drag the **Health_Pickup** object from the **Hierarchy** pane into the **Prefab** folder.

Refer to the following screenshot for an example of what the finished product should look like:

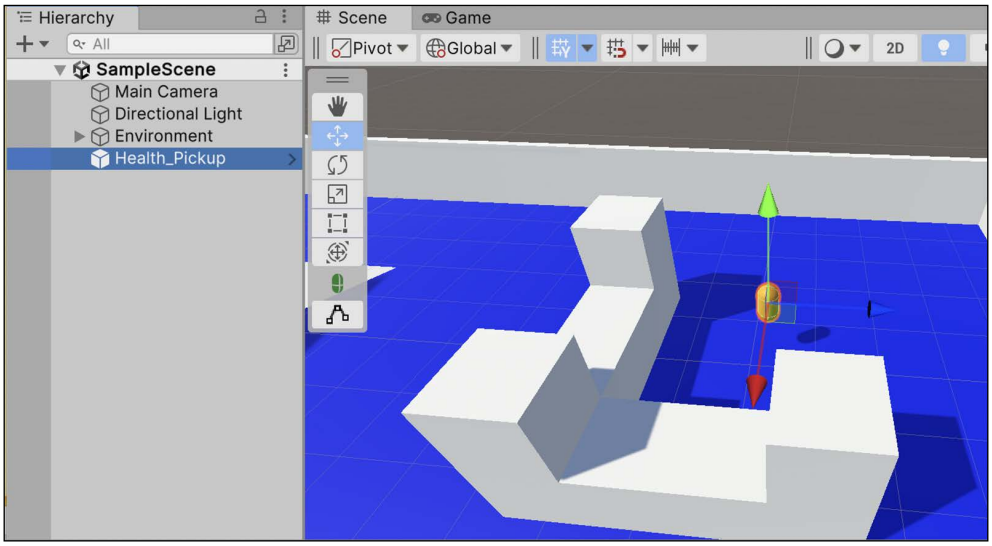

Figure 6.21: Pickup item and barrier Prefab in Scene

That wraps up our work with level design and layout for now. Next up, you're going to get a crash course in lighting with Unity, and we'll learn about animating our item later on in the chapter.

Lighting basics

Lighting in Unity is a broad topic, but it can be boiled down into two categories: real-time and precomputed. Both types of lights take into account properties such as the color and intensity of the light, as well as the direction it is facing in the scene, which can all be configured in the **Inspector** pane. The difference is how the Unity engine computes how the lights act:

- **Real-time lighting** is computed in every frame, meaning that any object that passes in its path will cast realistic shadows and generally behave like a real-world light source. However, this can significantly slow down your game and cost an exponential amount of computing power, depending on the number of lights in your scene.
- **Precomputed lighting**, on the other hand, stores the scene's lighting in a texture called a **lightmap**, which is then applied, or baked, into the scene. While this saves computing power, baked lighting is static. This means that it doesn't react realistically or change when objects move in the scene.

There is also a mixed type of lighting called **Precomputed Realtime Global Illumination**, which bridges the gap between real-time and precomputed processes. This is an advanced Unity-specific topic, so we won't cover it in this book, but feel free to view the documentation at: `https://docs.unity3d.com/Manual/GIIntro.html`.

Let's now take a look at how to create light objects in the Unity scene itself.

Creating lights

By default, every scene comes with a directional light component to act as a main source of illumination, but lights can be created in the hierarchy like any other GameObject. Even though the idea of controlling light sources might be new to you, they are objects in Unity, which means they can be positioned, scaled, and rotated to fit your needs:

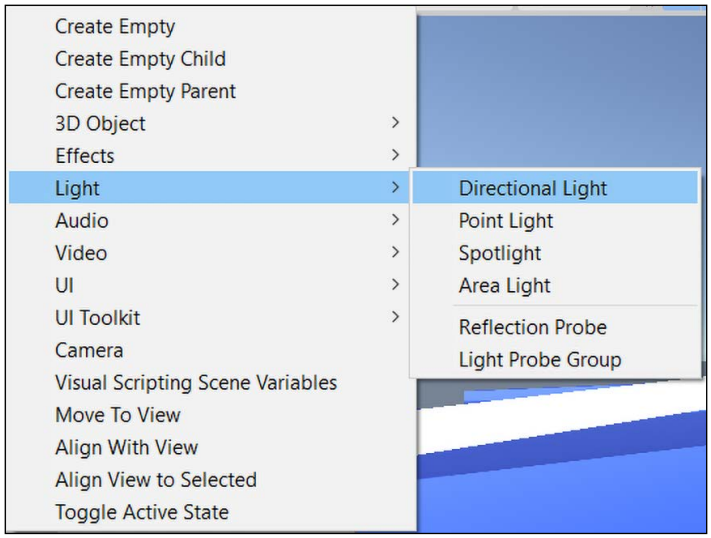

Figure 6.22: Lighting creation menu option

Let's take a look at some examples of real-time light objects and their performance:

- **Directional lights** are great for simulating natural light, such as sunshine. They don't have an actual position in the scene, but their light hits everything as if it's always pointed in the same direction.
- **Point lights** are essentially floating globes, sending light rays out from a central point in all directions. These have defined positions and intensities in the scene.
- **Spotlights** send light out in a given direction, but they are locked in by their angle and focused on a specific area of the scene. Think of these as spotlights or floodlights in the real world.
- **Area lights** are shaped like rectangles, sending out the light from their surface from a single side of the rectangle.

Reflection Probes and **Light Probe Groups** are beyond what we need for *Hero Born*; however, if you're interested, you can find out more at: `https://docs.unity3d.com/Manual/ReflectionProbes.html` and `https://docs.unity3d.com/Manual/LightProbes.html`.

Like all GameObjects in Unity, lights have properties that can be adjusted to give a scene a specific ambiance or theme.

Light component properties

The following screenshot shows the **Light** component on the directional light in our scene. All of these properties can be configured to create immersive environments, but the basic ones we need to be aware of are **Color**, **Mode**, and **Intensity**. These properties govern the light's tint, real-time or computed effects, and general strength:

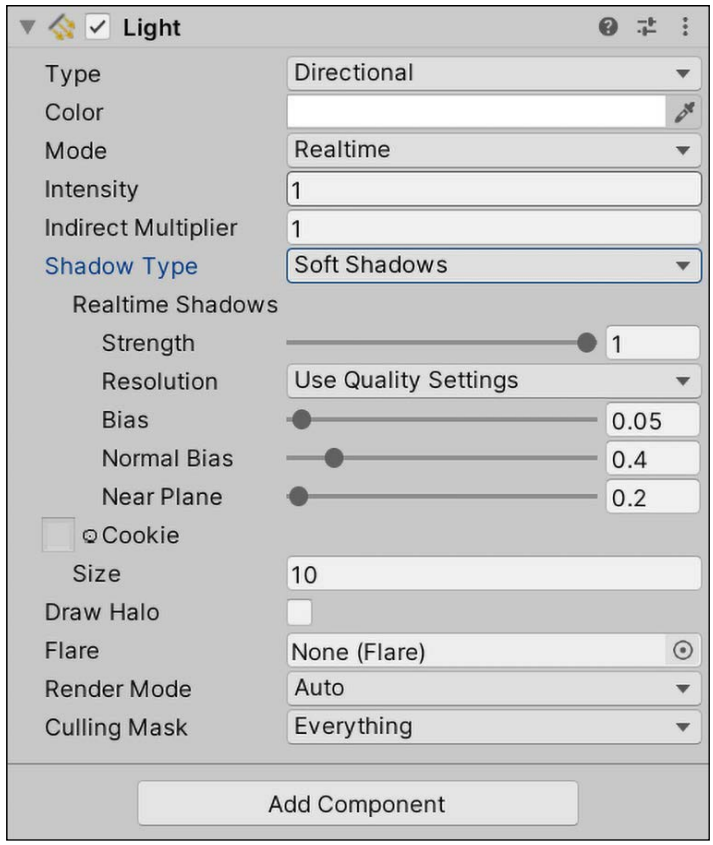

Figure 6.23: Light component in the Inspector window

Like other Unity components, these properties can be accessed through scripts and the `Light` class, which can be found at: `https://docs.unity3d.com/ScriptReference/Light.html`.

Try this out for yourself by selecting **+** | **Light** | **Point Light** and seeing how it affects the area lighting. After you've played around with the settings, delete the point light by right-clicking on it in the **Hierarchy** panel and choosing **Delete**.

Now that we know a little more about what goes into lighting up a game scene, let's turn our attention to adding some animations!

Animating in Unity

Animating objects in Unity can range from a simple rotation effect to complex character movements and actions. You can create animations in code or with the Animation and Animator windows:

- The **Animation** window is where animation segments, called clips, are created and managed using a timeline. Object properties are recorded along this timeline and are then played back to create an animated effect.
- The **Animator** window manages these clips and their transitions using objects called animation controllers.

You can find more information about the **Animator** window and its controllers at: `https://docs.unity3d.com/Manual/AnimatorControllers.html`.

Creating and manipulating your target objects in clips will have your game moving in no time. For our short trip into Unity animations, we'll create the same rotation effect in code, and by using the Animator.

Creating animations in code

To start, we're going to create an animation in code to rotate our health item pickup. Since all GameObjects have a `Transform` component, we can grab our item's `Transform` component and rotate it indefinitely.

To create an animation in code, you need to perform the following steps:

1. In the `Scripts` folder, create a new C# script named `ItemRotation`, and open it in Visual Studio Code.
2. At the top of the new class, add a public `int` variable containing the value `100` called `RotationSpeed`, and a private `Transform` variable called `_itemTransform`. If you don't specify an access level, Visual Studio assumes it's private, but the best practice is to use explicit access modifiers to make sure your code is crystal clear:

```
public int RotationSpeed = 100;
private Transform _itemTransform;
```

3. Inside the `Start()` method body, grab the GameObject's `Transform` component and assign it to `itemTransform`:

```
_itemTransform = this.GetComponent<Transform>();
```

4. Inside the `Update()` method body, call `_itemTransform.Rotate`. This `Transform` class method takes in three axes, one for the *x*, *y*, and *z* rotations you want to execute. Since we want the item to rotate end over end, we'll use the *x* axis and leave the others set to `0`:

```
_itemTransform.Rotate(RotationSpeed * Time.deltaTime, 0, 0);
```

You'll notice that we're multiplying our `RotationSpeed` by something called `Time.deltaTime`. This is the standard way of normalizing movement effects in Unity so that they look smooth no matter how fast or slow the player's computer is running. In general, you should always multiply your movement or rotation speeds by `Time.deltaTime`.

5. Back in Unity, select the `Health_Pickup` object in the `Prefabs` folder in the **Projects** pane and scroll down to the bottom of the **Inspector** window. Click **Add Component**, search for the `ItemRotation` script, and then press *Enter*:

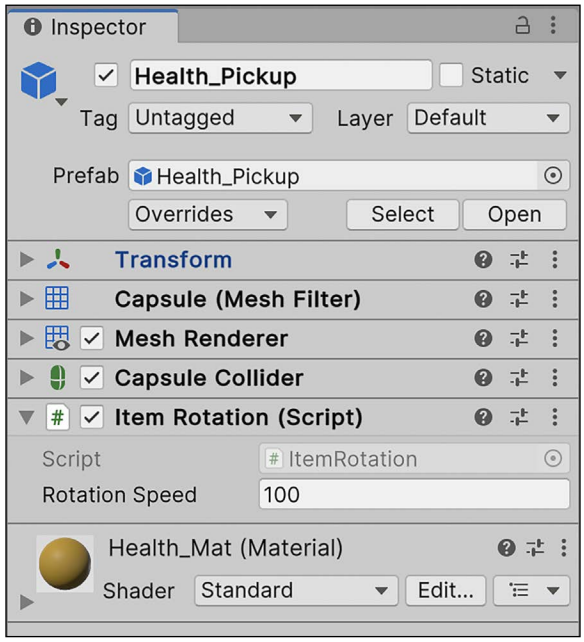

Figure 6.24: Add Component button in the Inspector panel

6. Now that our Prefab is updated, move the **Main Camera** so that you can see the `Health_Pickup` object and click on **Play**!

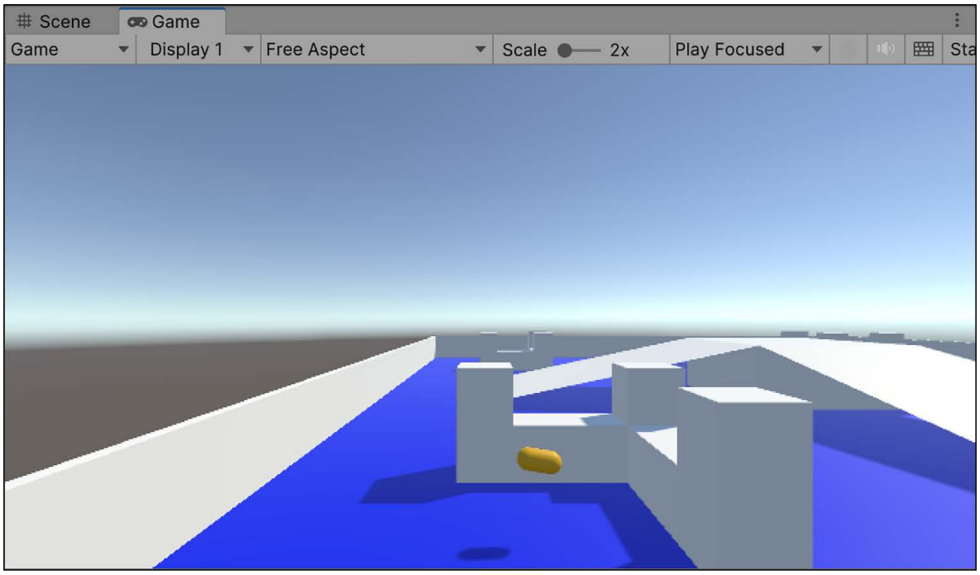

Figure 6.25: Screenshot of the camera focused on the health item

As you can see, the health pickup now spins around its *x* axis in a continuous and smooth animation! Now that you've animated the item in code, we'll duplicate our animation using Unity's built-in animation system.

Creating animations in the Unity Animation window

Before we go any further, it's important to choose the code method **OR** Unity's animation system for *a single animation*—not both. Otherwise, the two systems will end up fighting each other.

Any GameObject that you want to apply an animation clip to needs to be attached to an **Animator** component with an **Animation Controller** set. If there is no controller in the project when a new clip is created, Unity will create one and save it in the **Project** panel, which you can then use to manage your clips. Your next challenge is to create a new animation clip for the pickup item.

We're going to start animating the `Health_Pickup` Prefab by creating a new animation clip, which will spin the object around in an infinite loop. To create a new animation clip, we need to perform the following steps:

1. Navigate to **Window > Animation > Animation** and drag and drop the **Animation** tab next to the **Game** tab:

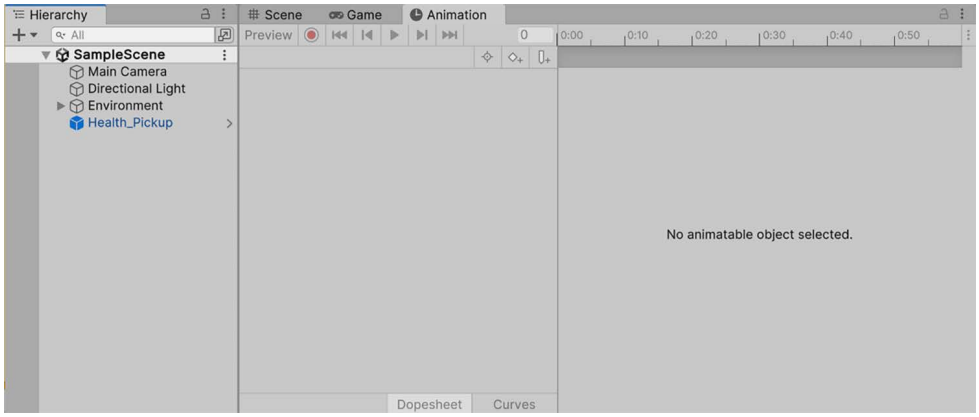

Figure 6.26: Screenshot of the Unity Animation window

2. Make sure the `Health_Pickup` item is selected in **Hierarchy** and then click on **Create** in the **Animation** panel:

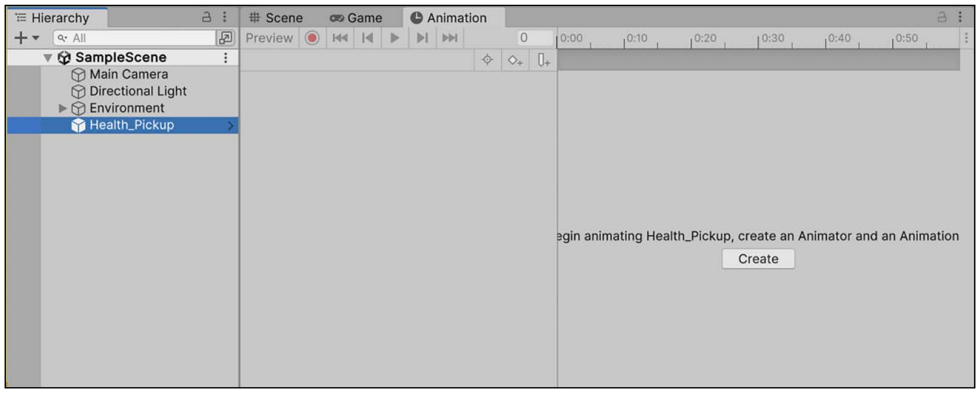

Figure 6.27: Screenshot of the Unity Animation window

3. Create a new folder under **Assets** from the following drop-down list, name it `Animations`, and then name the new clip `Spin`:

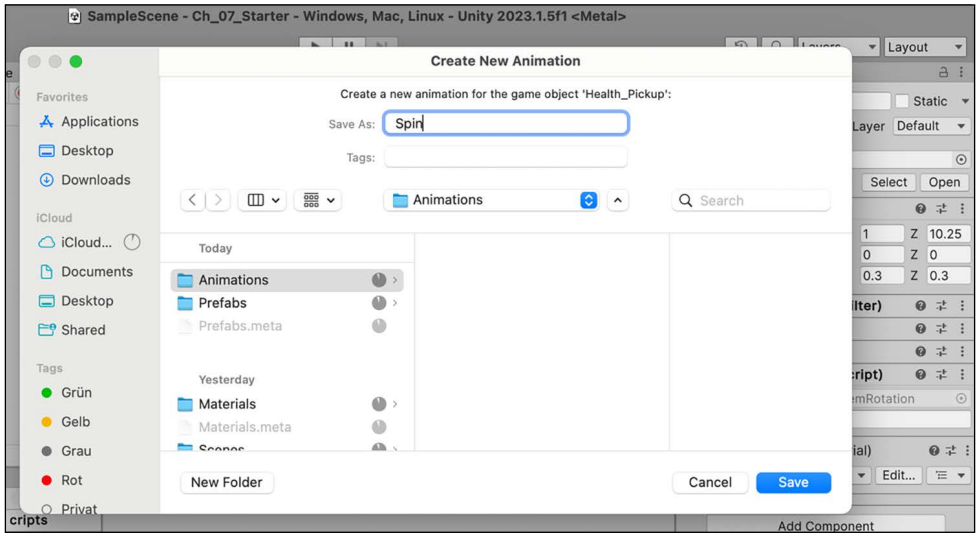

Figure 6.28: Screenshot of the Create New Animation window

4. Make sure the new clip shows up in the **Animation** panel:

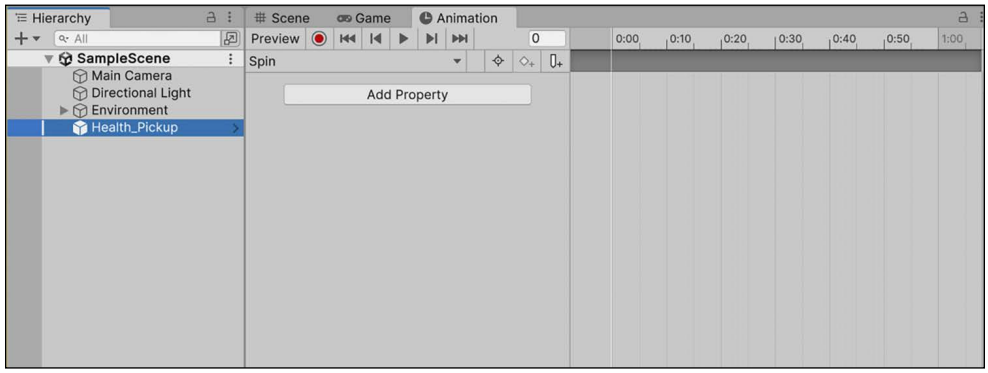

Figure 6.29: Screenshot of the Animation window with a clip selected

5. Since we didn't have any **Animator** controllers, Unity created one for us in the `Animation` folder called **Health_Pickup** (along with the animation itself).

    a. With **Health_Pickup** selected, note in the **Inspector** pane that when we created the clip, an **Animator** component was also added to the Prefab for us but hasn't been officially saved to the Prefab yet with the **Health_Pickup** controller set.

6. In the **Inspector**, notice that the + icon is showing in the top left of the **Animator** component, meaning it's not yet part of the **Health_Pickup** Prefab:

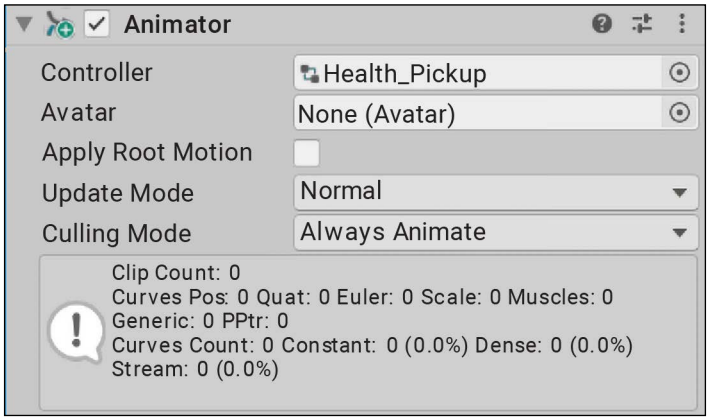

Figure 6.30: Animator component in the Inspector panel

7. Select the three-vertical-dots icon at the top right and choose **Added Component** > **Apply to Prefab 'Health_Pickup'**:

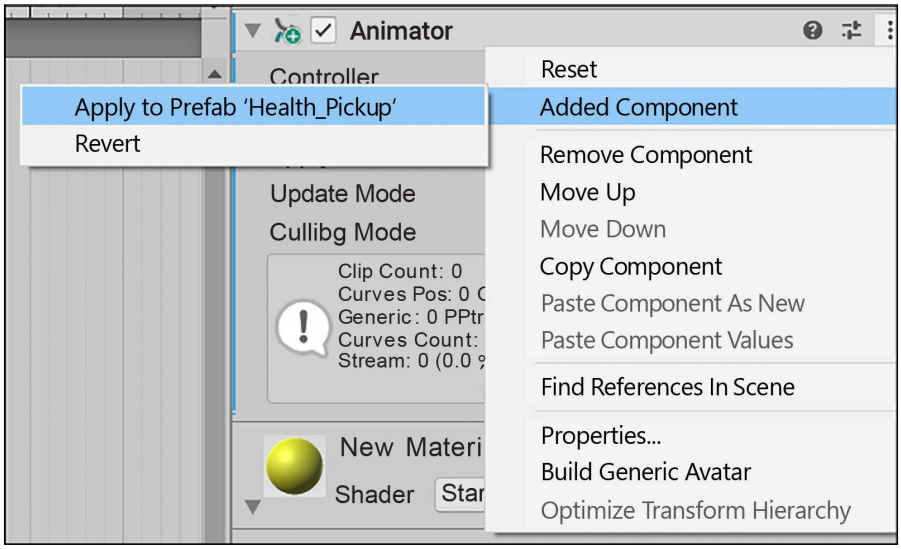

Figure 6.31: Screenshot of a new component being applied to the Prefab

Now that you've created and added an **Animator** component to the **Health_Pickup** Prefab, it's time to start recording some animation frames.

When you think of motion clips, as in movies, you may think of frames. As the clip moves through its frames, the animation advances, giving the effect of movement. It's no different in Unity; we need to record our target object in different positions throughout different frames so that Unity can play the clip.

Recording keyframes

Now that we have a clip to work with, you'll see a blank timeline in the **Animation** window. Essentially, when we modify our **Health_Pickup** Prefab's *z* rotation, or any other property that can be animated, the timeline will record those changes as keyframes. Unity then assembles those keyframes into your complete animation, similar to how individual frames on analog film play together into a moving picture.

Take a look at the following screenshot and remember the locations of the **Record** button and the timeline:

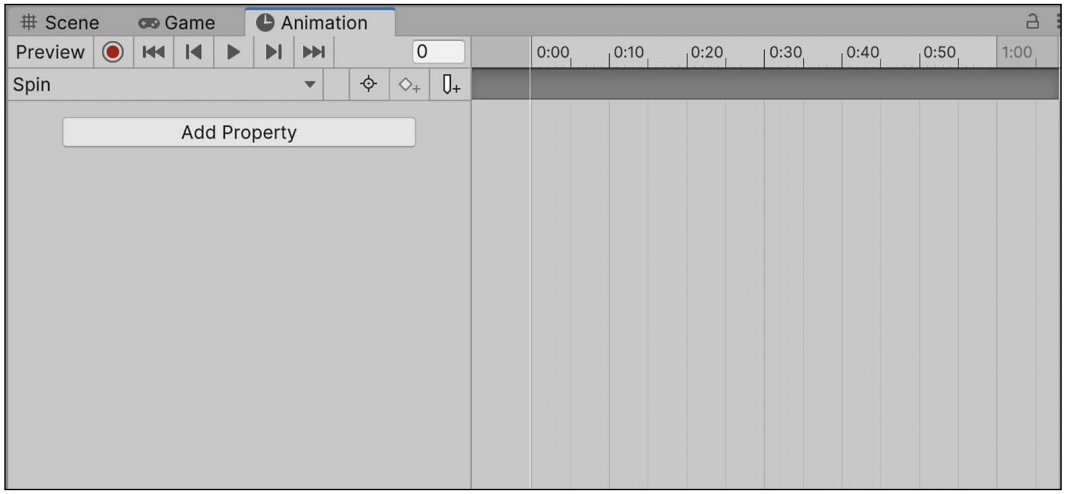

Figure 6.32: Screenshot of the Animation window and keyframe timeline

Now, let's get our item spinning. For the spinning animation, we want the **Health_Pickup** Prefab to make a complete 360-degree rotation on its *z* axis every second, which can be done by setting three keyframes and letting Unity take care of the rest:

1. Select the **Health_Pickup** object in the **Hierarchy** window, choose **Add Property** > **Transform**, and then click on the **+** sign next to **Rotation**:

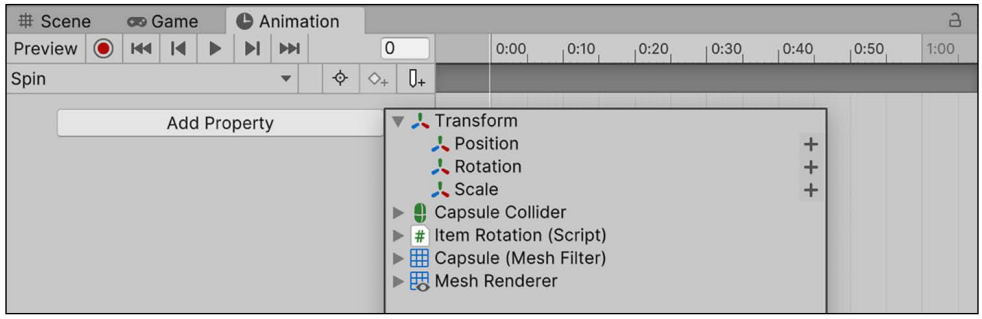

Figure 6.33: Screenshot of adding a Transform property for animation

2. Click on the **Record** button to start the animation:

    - Place your cursor at `0:00` on the timeline but leave the `Health_Pickup` Prefab's *z* rotation at `0` in the **Inspector**—the modifiable fields will appear in red so you don't accidentally animate a property by mistake:

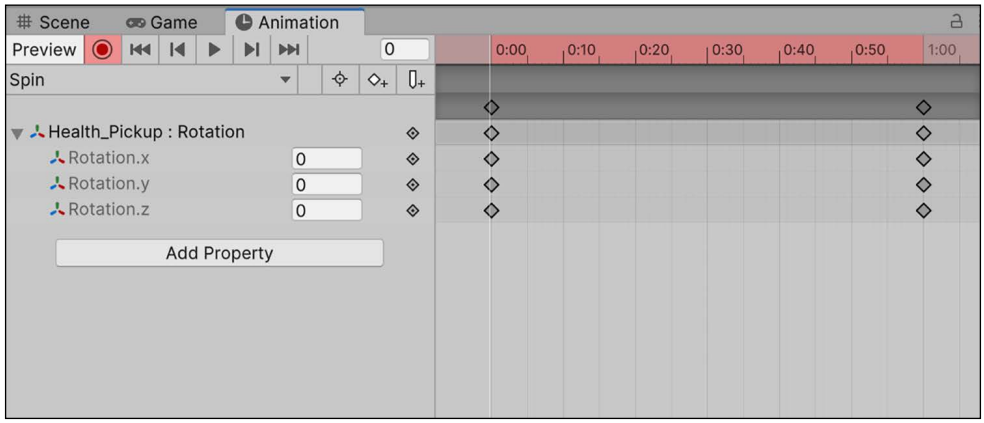

Figure 6.34: Screenshot of animatable property in the Inspector

- Place your cursor at `0:30` on the timeline and set the *z* rotation to `180`:

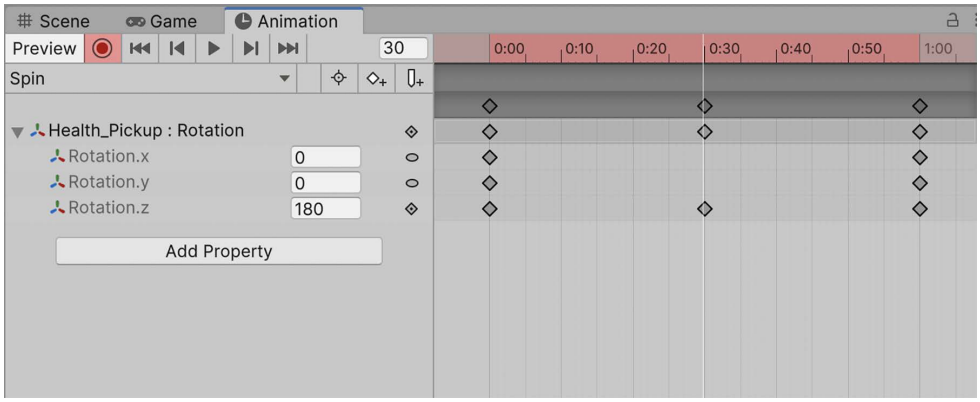

Figure 6.35: Screenshot of animatable property in the Inspector

- Place your cursor at `1:00` on the timeline and set the *z* rotation to `360`:

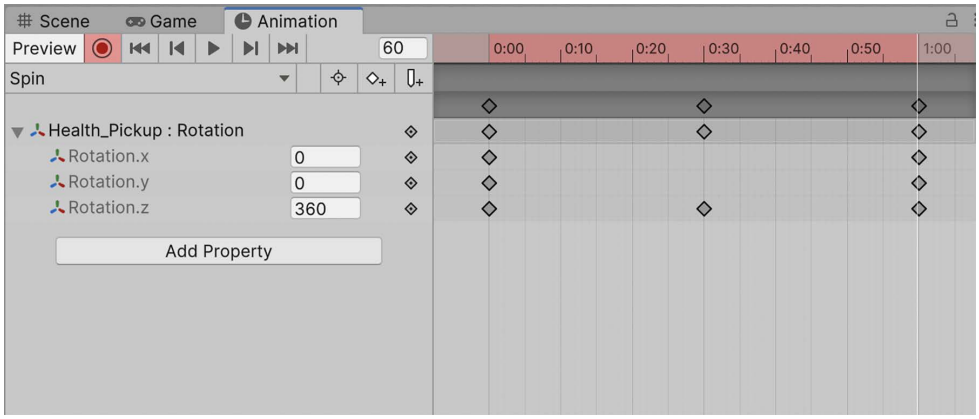

Figure 6.36: Screenshot of Animation keyframes being recorded

3. Click on the **Record** button to finish the animation.
4. Click on the **Play** button to the right of the **Record** button to see the animation loop:

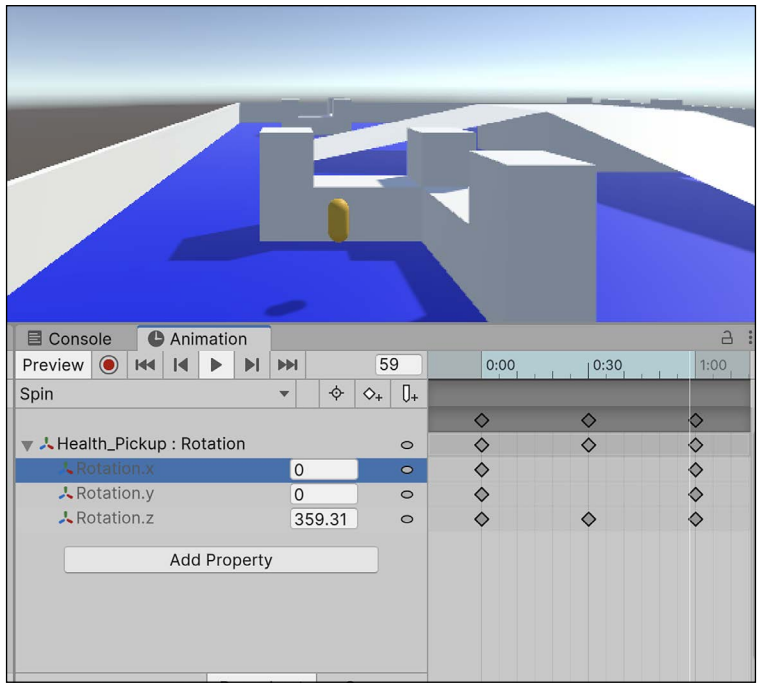

Figure 6.37: Screenshot of the animation playing

You'll notice that our **Animator** animation overrides the one we wrote in code earlier. Don't worry; this is expected behavior. You can click the small checkbox to the right of any component in the **Inspector** panel to activate or deactivate it. If you deactivate the **Animator** component, **Health_Pickup** will rotate around the *x* axis again using our code.

The **Health_Pickup** object now rotates on the *z* axis between 0, 180, and 360 degrees every second, creating the looping spin animation. If you play the game now, the animation will run indefinitely.

All animations have curves, which determine specific properties of how an animation executes. We won't be doing too much with these, but it's important to understand the basics.

Curves and tangents

In addition to animating an object property, Unity lets us manage how the animation plays out over time with animation curves. So far, we've been in **Dopesheet** mode, which you can change at the bottom of the **Animation** window.

If you click on the **Curves** view (pictured in the following screenshot), you'll see a different graph with accent points in place of our recorded keyframes.

We want the spinning animation to be smooth—what we call linear—so we'll leave everything as is. However, speeding up, slowing down, or altering the animation at any point in its run can be done by dragging or adjusting the points on the curve graph in any direction:

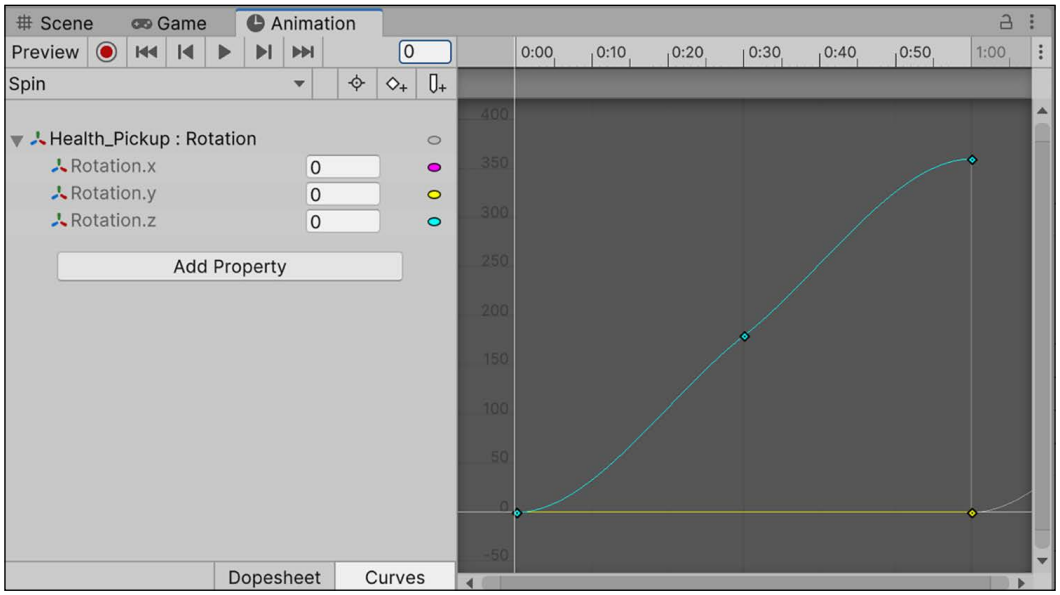

Figure 6.38: Screenshot of an animation playing in the Animation window

With animation curves handling how properties act over time, we still need a way to fix the stutter that occurs every time the **Health_Pickup** animation repeats. For that, we need to change the animation's tangent, which manages how keyframes blend into one another.

These options can be accessed by right-clicking on any top keyframe on the timeline in **Dopesheet** mode, which you can see here:

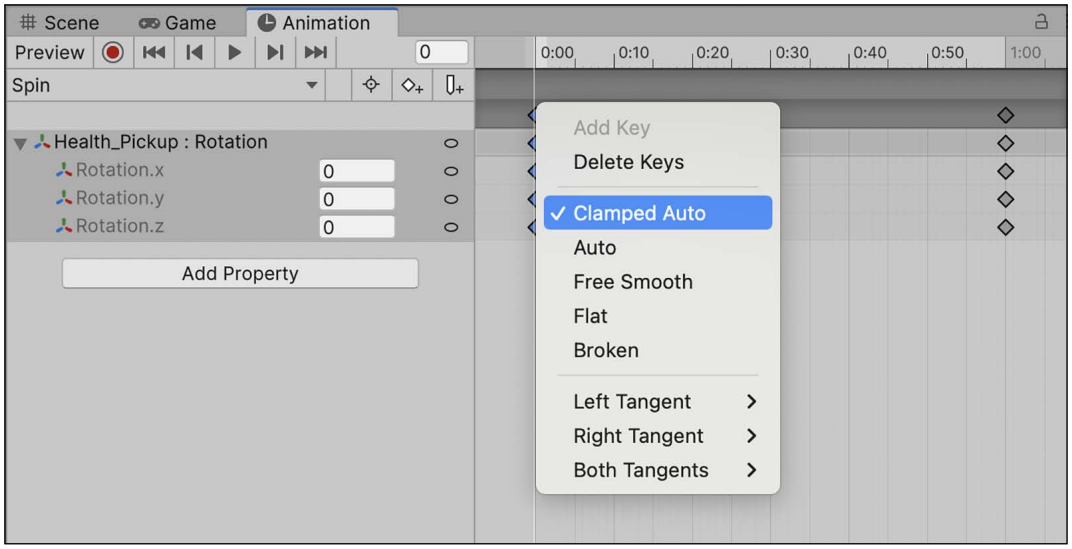

Figure 6.39: Screenshot of keyframe smoothing options

Both curves and tangents are intermediate/advanced, so we won't be delving too deeply into them. If you're interested, you can take a look at the documentation on animation curves and tangent options at: `https://docs.unity3d.com/Manual/animeditor-AnimationCurves.html`.

If you play the spinning animation as it is now, there's a slight pause between when the item completes its full rotation and starts a new one. Your job is to smooth that out, which is the subject of the next challenge.

Let's adjust the tangents on the first and last frames of the animation so that the spinning animation blends seamlessly together when it repeats:

1. Right-click on the first and last keyframes' diamond icons on the animation timeline and select **Auto**:

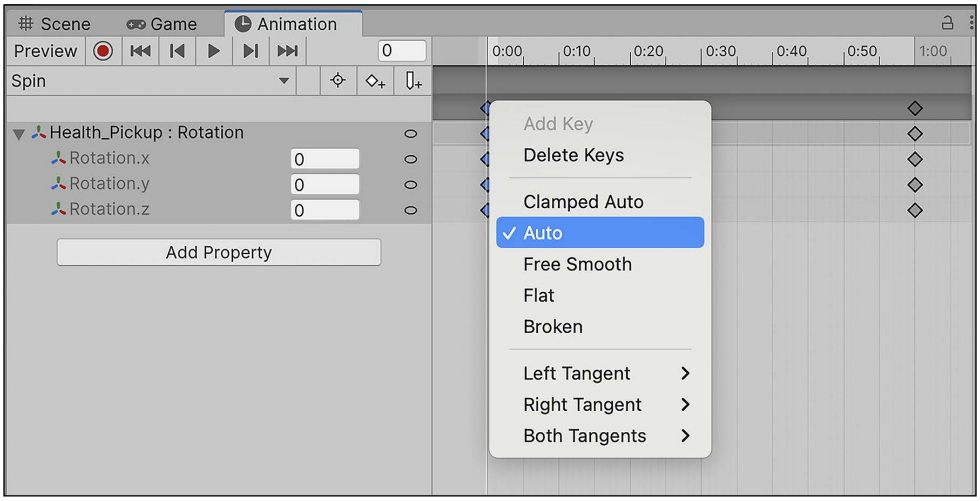

Figure 6.40: Changing keyframe smoothing options

2. If you haven't already done so, move the **Main Camera** so that you can see the `Health_Pickup` object, and click on **Play**.

Changing the first and last keyframe tangents to **Auto** tells Unity to make their transitions smooth, which eliminates the jerky stop/start motion when the animation loops.

That's all the animation you'll need for this book, but I'd encourage you to check out the full toolbox that Unity offers in this area. Your games will be more engaging and your players will thank you!

Summary

We made it to the end of another chapter that had a lot of moving parts, which might have been a lot for those of you who are new to Unity.

Even though this book is focused on the C# language and its implementation in Unity, we still need to take time to get an overview of game development, documentation, and the non-scripting features of the engine. While we didn't have time for in-depth coverage of the lighting and animation, it's worth getting to know them if you're thinking about continuing to create Unity projects.

In the next chapter, we'll be switching our focus back to programming *Hero Born*'s core mechanics, starting with setting up a moveable player object, controlling the camera, and understanding how Unity's physics system governs the game world.

Pop quiz—basic Unity features

1. Cubes, capsules, and spheres are examples of what kind of GameObject?
2. What axis does Unity use to represent depth, which gives scenes their 3D appearance?
3. How do you turn a GameObject into a reusable Prefab?
4. What unit of measurement does the Unity animation system use to record object animations?

Don't forget to check your answers against mine in the *Pop Quiz Answers* appendix to see how you did!